



Robinson Aerospace Systems Pty Ltd

Contact: edward@robinson-aerospace.com

V0.1 – August 2026

RASCube V2 Communication Chain

Abstract

This document describes the implemented RASCubeV2 command and telemetry path from a browser UI or external program, through the USB receiver and radio link, to the OBC STM and Arduino. It defines and catalogues the supported commands, and documents the current source behaviour.

Contents

1	Introduction	3
1.1	Scope	3
1.2	Notation and Encoding	3
2	Host to USB Receiver Transport	3
3	Command Routing	4
4	Uplink Ports	4
4.1	USB Receiver	4
4.1.1	USB Information Request (Port 0x00)	4
4.1.2	USB Serial-Number Filter (Port 0x01)	5
4.1.3	USB RF Configuration (Port 0x02)	5
4.1.4	USB Mode Request (Port 0x03)	5
4.1.5	USB Bootloader Enter (Port 0x0A)	5
4.1.6	USB Bootloader Command (Port 0x0B)	5
4.1.7	USB Bootloader Write (Port 0x0C)	5
4.2	OBC STM	6
4.2.1	OBC Information Request (Port 0x12)	6
4.2.2	OBC Camera Request (Port 0x13)	6
4.2.3	OBC RF Configuration (Port 0x14)	6
4.2.4	OBC Send Data While Camera Active (Port 0x15)	6
4.2.5	OBC Set Flash Settings (Port 0x16)	6
4.2.6	OBC Flash Settings Request (Port 0x17)	6
4.3	OBC Arduino and Add-ons	6
4.3.1	Arduino Set RGB (Port 0x80)	7
4.3.2	Arduino Buzzer Control (Port 0x81)	7
4.3.3	Arduino Start Calibration (Port 0x82)	7
4.3.4	Arduino Set Calibration (Port 0x83)	7
4.3.5	Arduino Play Startup Song (Port 0x84)	7
4.3.6	Arduino Get Add-ons Request (Port 0x85)	7
4.3.7	Arduino Add-on Command (Port 0x86)	7
4.3.8	Arduino Get Add-on Status Request (Port 0x87)	8
4.3.9	Arduino Add-on Information Request (Port 0x88)	8
5	Downlink Ports	8
5.1	USB Receiver	8
5.1.1	USB Information Response (Port 0x00)	8
5.1.2	USB Mode Response (Port 0x01)	9
5.1.3	USB Bootloader Command Response (Port 0x0A)	9
5.2	OBC STM	9
5.2.1	Main OBC Telemetry (Port 0x10)	9
5.2.2	OBC Information Response (Port 0x12)	10
5.2.3	OBC Legacy Camera Data (Port 0x13)	10
5.2.4	OBC Flash Settings Response (Port 0x14)	10

5.2.5	OBC JPEG Camera Data (Port 0x15)	10
5.3	OBC Arduino and Add-ons	10
5.3.1	User Telemetry (Port 0x11)	10
5.3.2	Arduino Calibration Status (Port 0x80)	11
5.3.3	Arduino User-Data Name (Port 0x82)	11
5.3.4	Arduino User-Data Item Count (Port 0x83)	11
5.3.5	Arduino Enabled Add-ons Response (Port 0x86)	11
5.3.6	Arduino Add-on Telemetry (Port 0x87)	11
5.3.7	Arduino Add-on Status (Port 0x88)	11
5.3.8	Arduino Add-on Information Response (Port 0x89)	11
5.4	Multi-Source Status	11
5.4.1	RASCube Status (Port 0xF0)	11
6	User Telemetry and Custom Commands	11
7	Add-ons	12
7.1	Common Structure	12
7.2	Advanced Sensor Module	13
7.2.1	Host Commands	13
7.2.2	Telemetry and Status	13
7.2.3	I ² C Register Interface	13
7.3	Environmental Sensor	14
7.4	Reaction Wheel	14
7.4.1	Host-Level Control	14
7.4.2	Expanded I ² C Contract	15
8	Writing an External Client	15
8.1	Host Requirements	15
8.2	Direct Radio Implementations	15
9	Limitations and Troubleshooting	15
9.1	Asynchronous Receiver Status	15

1 Introduction

1.1 Scope

The scope is the RASCubeV2 path shown in Figure 1.

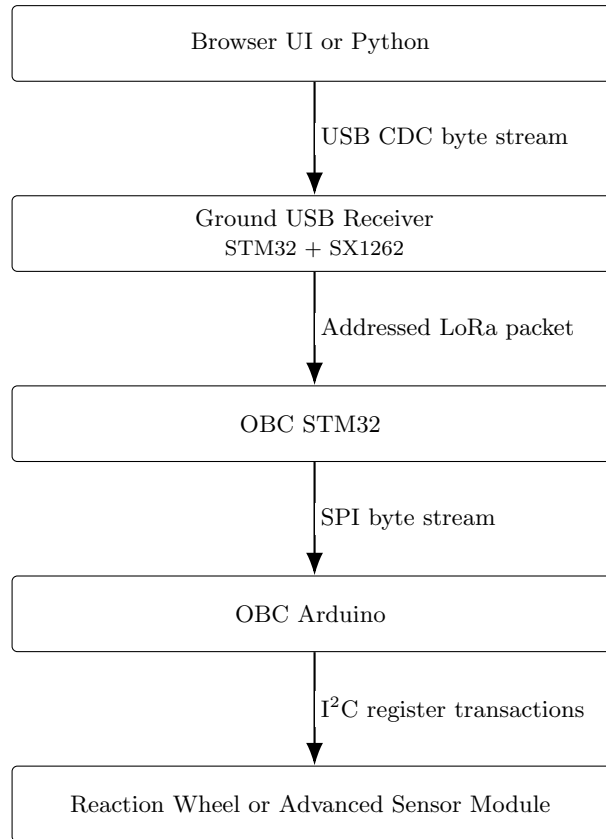


Figure 1: Communication flow between the ground interface and on-board modules.

The browser UI talks directly to the receiver using the Web Serial API. There is no web server or HTTP service in the command path. A Python application can therefore replace the UI by opening the same USB CDC device and implementing the framing in Section 2.

1.2 Notation and Encoding

Unless stated otherwise:

- Integers containing more than one byte are little-endian.
- Signed integers use two’s-complement representation.
- A `float32` is an IEEE-754, 32-bit, little-endian floating-point value.
- `N` is the payload length in bytes and excludes its enclosing header.
- A “port” is the one-byte command or data-stream identifier. Port values are reused by direction, so a number alone is not globally unique.
- An “uplink” is a host-to-device frame. This includes commands consumed by the ground USB receiver and commands forwarded over radio to the OBC, Arduino, or an add-on.
- A “downlink” is a device-to-host frame. This includes solicited responses and asynchronous telemetry, status, camera, and metadata packets.
- Uplink and downlink are always stated relative to the host. A receiver-local uplink does not cross the radio link, but it is still host-originated traffic.
- Hexadecimal wire examples include every byte transmitted at the host USB framing layer unless stated otherwise.

2 Host to USB Receiver Transport

The USB CDC connection is treated as a continuous byte stream.

Offset	Size	Type	Units	Meaning
0	1	u8	identifier	Port or command ID
1	1	u8	bytes	Payload length N
2	N	$N \times \text{u8}$	bytes	Raw payload

There is no preamble, terminator, escaping, sequence number, or checksum. The parser must retain partial state until all N bytes have arrived. If a byte is lost, there is no in-band resynchronization marker; closing and reopening the CDC connection resets the receiver parser and queues.

The UI requests 1,000,000 baud, 8 data bits, no parity, one stop bit, and no flow control. The receiver is USB CDC and ignores CDC line-coding values, but external clients should use the UI values for consistency. The normal V2 receiver identifies as USB VID 0x0483, PID 0x5740. DTR transitions are significant: the receiver uses them to detect connection changes, clears communication buffers, and resets its radio configuration to defaults. The client should select the satellite serial number again after every reconnect.

The firmware length field can represent 255 bytes and the UI restricts its outbound payload to 253 bytes. A command forwarded over radio must not exceed 247 bytes because of the LoRa headers. A portable host implementation should therefore enforce 247 bytes for all non-local commands.

Although the receiver application accepts a zero-length USB frame, the UI receive parser, and both OBC and Arduino SPI parsers have zero-length edge cases. Existing no-argument commands conventionally carry one dummy byte, normally 00. New host software should follow that convention.

3 Command Routing

The receiver and OBC use switch-based dispatch. Receiver-local ports are consumed before radio forwarding. OBC local ports are consumed before Arduino forwarding. Therefore, an Arduino command must use a port that is not intercepted by an earlier layer.

Most write commands do not produce a response. The existence of a response for one command does not imply a generic ACK mechanism. A client should apply its own timeout to requests and must regard a timeout as ambiguous: the command may have been lost, ignored, delayed, or may have no response by design.

4 Uplink Ports

Every port in this section is transmitted by the host. Commands are grouped by the component that consumes them; the receiver forwards commands not consumed locally.

4.1 USB Receiver

The following commands are consumed locally by the USB receiver.

Port	Name	Payload	Behavior
0x00	USB_INFO_REQ	One dummy byte	Returns receiver information on 0x00.
0x01	USB_SN_FILTER	serialNumber:u32LE	Selects channel and radio address asynchronously; no response.
0x02	USB_SET_RF_CFG	V2 UI: SF, BW; some firmware: SF, BW, CR	Changes receiver modulation; no response.
0x03	USB_MODE_REQ	One dummy byte	Returns mode on port 0x01; normal application mode is 0x02.
0x0A	USB_BOOTLOADER_ENTER	Payload ignored	Stores the bootloader opcode and resets.
0x0B	USB_BOOTLOADER_CMD	Five bytes	Bootloader-mode erase and upload control; responds on 0x0A.
0x0C	USB_BOOTLOADER_WRITE	128 data bytes	Bootloader-mode block write; responds on 0x0A.
Other	Forwarded endpoint	Up to 247 bytes recommended	Queued for transmission to the OBC.

4.1.1 USB Information Request (Port 0x00)

Direction: Uplink, host to USB receiver.

Requests the receiver firmware identity. Send one dummy u8, conventionally zero; the complete frame is 00 01 00. A successful request produces the USB Information Response on port 0x00.

4.1.2 USB Serial-Number Filter (Port 0x01)

Direction: Uplink, host to USB receiver.

Selects the satellite radio address and channel asynchronously. The payload is `serialNumber:u32LE`; serial number 1 is 01 04 01 00 00 00. No response or acknowledgement packet is expected.

4.1.3 USB RF Configuration (Port 0x02)

Direction: Uplink, host to USB receiver.

Changes the receiver spreading factor and bandwidth. The V2 UI payload is `spreadingFactor:u8`, `bandwidthIndex:u8`; for SF 7 and 125 kHz it sends 02 02 07 07. Receiver firmware variants may accept a third `codingRateIndex:u8`. No response or acknowledgement packet is expected.

Bandwidth and coding-rate values are indices:

BW index	0	1	2	3	4	5	6	7	8	9
BW (kHz)	7	10	15	20	31	41	62	125	250	500

CR index	0	1	2	3
Coding rate	4/5	4/6	4/7	4/8

The OBC implementation currently programs coding rate 4/5. Setting the receiver to a different coding rate can therefore break the link. Changing receiver and OBC RF settings is not transactional and neither command returns an ACK.

4.1.4 USB Mode Request (Port 0x03)

Direction: Uplink, host to USB receiver.

Requests the receiver operating mode. Send one dummy u8; the complete frame is 03 01 00. A successful request produces the USB Mode Response on port 0x01.

4.1.5 USB Bootloader Enter (Port 0x0A)

Direction: Uplink, host to USB receiver.

Stores the bootloader-entry opcode and resets the receiver. The UI sends a dummy u8, giving 0A 01 00. There is no immediate acknowledgement; reconnect and use USB Mode Request to confirm bootloader mode.

4.1.6 USB Bootloader Command (Port 0x0B)

Direction: Uplink, host to USB receiver bootloader.

Controls an already-running receiver bootloader. Its five-byte payload is `opcode:u8`, `argument[4]:u8`. The defined forms are:

Opcode	Argument bytes	Complete frame example
0x00	<code>numPages:u16LE</code> , 00, 00	0B 05 00 20 00 00 00 erases 32 pages
0x01	Four zero bytes	0B 05 01 00 00 00 00 starts upload
0x02	Four zero bytes	0B 05 02 00 00 00 00 completes upload

Each successful operation produces the corresponding USB Bootloader Command Response on port 0x0A.

4.1.7 USB Bootloader Write (Port 0x0C)

Direction: Uplink, host to USB receiver bootloader.

Writes one 128-byte application block while the receiver is in programming mode. The payload is `applicationData[128]:u8`, so a frame starts 0C 80 followed by exactly 128 bytes. A successful write produces result 0x03 in the USB Bootloader Command Response.

Ports 0x0B and 0x0C are receiver bootloader commands only after the device has entered the bootloader. In normal application mode they are not consumed locally and would be forwarded toward the OBC.

4.2 OBC STM

The following commands are forwarded by the receiver and consumed by the OBC STM.

Port	Name	Payload	Behavior or response
0x12	OBC_INFO_REQ	Dummy byte recommended	Returns 26-byte information response on 0x12.
0x13	OBC_CAM_REQ	Dummy byte recommended	Starts capture if idle; returns JPEG blocks on 0x15.
0x14	OBC_SET_RF_CFG	SF:u8, BW-index:u8	Changes OBC spreading factor and bandwidth; no response.
0x15	OBC_SEND_DATA_WHILE_CAM	enable:u8	Controls normal telemetry during camera transfer; no response.
0x16	OBC_SET_FLASH_SETTINGS	One flags byte	Bit 0 controls startup sound; no response.
0x17	OBC_REQ_FLASH_SETTINGS	Dummy byte recommended	Returns one settings byte on 0x14.
Other	Arduino forwarding path	At least one byte	Queued unchanged to the Arduino over SPI.

4.2.1 OBC Information Request (Port 0x12)

Direction: Uplink, host to OBC STM through the USB receiver.

Requests the cached STM and Arduino firmware identities. Send one dummy u8; the complete frame is 12 01 00. A successful request produces the OBC Information Response on port 0x12.

4.2.2 OBC Camera Request (Port 0x13)

Direction: Uplink, host to OBC STM through the USB receiver.

Starts a camera capture when the camera is idle. Send one dummy u8; the complete frame is 13 01 00. Current firmware returns one or more OBC JPEG Camera Data packets on port 0x15; older firmware can instead return OBC Legacy Camera Data on port 0x13. There is no separate acceptance ACK.

4.2.3 OBC RF Configuration (Port 0x14)

Direction: Uplink, host to OBC STM through the USB receiver.

Changes the OBC spreading factor and bandwidth. Its payload is `spreadingFactor:u8, bandwidthIndex:u8`; SF 7 and 125 kHz is 14 02 07 07. Valid bandwidth indices are 0–9 as listed under USB RF Configuration. No response or acknowledgement packet is expected.

4.2.4 OBC Send Data While Camera Active (Port 0x15)

Direction: Uplink, host to OBC STM through the USB receiver.

Controls whether normal telemetry continues during a camera transfer. The payload is `enable:u8`; 15 01 01 enables it and 15 01 00 disables it. No response or acknowledgement packet is expected.

4.2.5 OBC Set Flash Settings (Port 0x16)

Direction: Uplink, host to OBC STM through the USB receiver.

Stores the OBC settings flags. The payload is `settings:u8`; bit 0 controls the startup sound, so 16 01 01 enables it. No response or acknowledgement packet is expected.

4.2.6 OBC Flash Settings Request (Port 0x17)

Direction: Uplink, host to OBC STM through the USB receiver.

Requests the stored OBC settings. Send one dummy u8; the complete frame is 17 01 00. A successful request produces the OBC Flash Settings Response on port 0x14.

4.3 OBC Arduino and Add-ons

These commands are reached only after receiver and OBC forwarding. Use a dummy byte for no-parameter commands.

Port	Name	Payload	Current behavior
0x80	ARD_SET_RGB	red, green, blue	Sets RGB values; no response.
0x81	ARD_BUZZER_CTL	No implemented format	Declared but not dispatched; discarded.
0x82	ARD_START_CALIB	Dummy byte	Starts gyro/magnetometer calibration; status events use 0x80.
0x83	ARD_SET_CALIB	Nine float32: gyro offsets[3], magnetometer offsets[3], magnetometer scales[3]	Applies runtime calibration using Arduino field order.
0x84	ARD_PLAY_STARTUP_SONG	Dummy byte	Queues the startup song; no response.
0x85	ARD_GET_ADDONS_REQ	Dummy byte	Returns an eight-byte add-on bitmap on 0x86.
0x86	ARD_ADDONS_DATA	Add-on envelope	Sends a command to an add-on handler.
0x87	ARD_GET_ADDONS_STATUS_REQ	Dummy byte	Requests status; currently only Advanced Sensor status is produced.
0x88	ARD_ADDONS_INFO_REQ	Dummy byte	Returns version records on 0x89; no response when no add-ons are enabled.

4.3.1 Arduino Set RGB (Port 0x80)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Sets the Arduino RGB outputs using red:u8, green:u8, blue:u8. Full-intensity red is 80 03 FF 00 00. No response or acknowledgement packet is expected.

4.3.2 Arduino Buzzer Control (Port 0x81)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Is reserved for buzzer control, but the current Arduino dispatcher does not implement a payload format and discards the command. No portable payload example or response is defined; clients should not send this port.

4.3.3 Arduino Start Calibration (Port 0x82)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Starts gyro and magnetometer calibration. Send one dummy u8; the complete frame is 82 01 00. Completion events arrive as Arduino Calibration Status packets on port 0x80; there is no immediate acceptance ACK.

4.3.4 Arduino Set Calibration (Port 0x83)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Applies runtime sensor calibration. The 36-byte payload contains nine float32 values in this order: gyro offsets X/Y/Z, magnetometer offsets X/Y/Z, then magnetometer scales X/Y/Z. A zero-offset, unit-scale frame begins 83 24, followed by 24 zero bytes and three 00 00 80 3F values. No response or acknowledgement packet is expected.

4.3.5 Arduino Play Startup Song (Port 0x84)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Queues the startup song. Send one dummy u8; the complete frame is 84 01 00. No response or acknowledgement packet is expected.

4.3.6 Arduino Get Add-ons Request (Port 0x85)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Requests the Arduino's cached enabled-add-on bitmap. Send one dummy u8; the complete frame is 85 01 00. A successful request produces the Arduino Enabled Add-ons Response on port 0x86.

4.3.7 Arduino Add-on Command (Port 0x86)

Direction: Uplink, host to an add-on through the receiver, OBC STM, and Arduino.

Routes a command to an add-on handler. The payload is addonId:u8, subcommandId:u8, subcommandPayload[:u8. For example, 86 03 02 00 00 starts recording on add-on 2. There is no generic response or ACK; any later telemetry or status is asynchronous and command-specific. Detailed subcommands are given in Section 7.

4.3.8 Arduino Get Add-on Status Request (Port 0x87)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Requests status from enabled add-ons. Send one dummy `u8`; the complete frame is `87 01 00`. Supporting devices produce one or more Arduino Add-on Status packets on port `0x88`; no generic completion packet is sent.

4.3.9 Arduino Add-on Information Request (Port 0x88)

Direction: Uplink, host to OBC Arduino through the receiver and OBC STM.

Requests firmware versions from enabled add-ons. Send one dummy `u8`; the complete frame is `88 01 00`. Results arrive in the Arduino Add-on Information Response on port `0x89`. No packet is returned when no add-ons are enabled.

Calibration status payload values are `0x01` for gyro complete and `0x02` for magnetometer complete. The OBC forwards a host `ARD_SET_CALIB` payload unchanged, so the payload must use the Arduino order shown above. This host command changes runtime Arduino calibration and does not update OBC flash. The STM declaration assigns different field names to the same byte offsets, causing mislabeled OBC diagnostics, although raw flash store/replay preserves the bytes.

5 Downlink Ports

Every port in this section is transmitted toward the host. Solicited responses identify their triggering uplink; asynchronous packets can arrive while any request is pending.

Port	Packet	Source	Delivery
0x00	USB Information Response	USB receiver	Solicited by port 0x00
0x01	USB Mode Response	USB receiver	Solicited by port 0x03
0x0A	USB Bootloader Command Response	USB receiver	Solicited by port 0x0B or 0x0C
0x10	Main OBC Telemetry	OBC STM	Asynchronous
0x11	User Telemetry	OBC Arduino	Asynchronous
0x12	OBC Information Response	OBC STM	Solicited by port 0x12
0x13	OBC Legacy Camera Data	OBC STM	Solicited stream from port 0x13
0x14	OBC Flash Settings Response	OBC STM	Solicited by port 0x17
0x15	OBC JPEG Camera Data	OBC STM	Solicited stream from port 0x13
0x80	Arduino Calibration Status	OBC Arduino	Asynchronous after port 0x82
0x82	Arduino User-Data Name	OBC Arduino	Asynchronous
0x83	Arduino User-Data Item Count	OBC Arduino	Asynchronous
0x86	Arduino Enabled Add-ons Response	OBC Arduino	Solicited by port 0x85
0x87	Arduino Add-on Telemetry	Add-on through Arduino	Asynchronous
0x88	Arduino Add-on Status	Add-on through Arduino	Solicited or asynchronous
0x89	Arduino Add-on Information Response	OBC Arduino	Solicited by port 0x88
0xF0	RASCube Status	Receiver, OBC STM, or Arduino	Asynchronous

5.1 USB Receiver

The following downlinks originate in the USB receiver or its bootloader.

5.1.1 USB Information Response (Port 0x00)

Direction: Downlink, USB receiver to host.

This is the solicited response to USB Information Request. The complete frame begins `00 0D`; its 13-byte payload is:

Offset	Size	Type	Units	Meaning
0	4	<code>u32LE</code>	version	Receiver software version
4	8	<code>8 × u8</code>	ASCII	Eight-character Git hash
12	1	<code>u8</code>	flag	Dirty build when nonzero

5.1.2 USB Mode Response (Port 0x01)

Direction: Downlink, USB receiver to host.

This is the solicited response to USB Mode Request. The payload is `mode:u8`; 01 01 02 reports normal application mode. Values 0x00, 0x01, and 0x02 mean bootloader, programming, and application.

5.1.3 USB Bootloader Command Response (Port 0x0A)

Direction: Downlink, USB receiver bootloader to host.

This solicited acknowledgement carries `result:u8`. Results 0x00, 0x01, 0x02, and 0x03 mean erase complete, upload started, upload complete, and block write complete. For example, 0A 01 03 acknowledges a successful USB Bootloader Write.

5.2 OBC STM

The following downlinks originate in the OBC STM.

5.2.1 Main OBC Telemetry (Port 0x10)

Direction: Downlink, OBC STM to host.

This asynchronous packet carries OBC state and sensor data. A current frame begins 10 79 and contains this packed 121-byte payload:

Offset	Size	Type	Units	Meaning
0	4	u32LE	count	Packet sequence number
4	2	u16LE	mV	Main 5 V rail
6	2	u16LE	mV	Main 3.3 V rail
8	2	u16LE	raw counts	Solar LDR channel 1
10	2	u16LE	raw counts	Solar LDR channel 2
12	2	u16LE	raw counts	Solar LDR channel 3
14	2	u16LE	mV	Battery-charger bus voltage
16	2	i16LE	mA	Battery charging current
18	2	i16LE	mV	USB bus voltage
20	2	i16LE	mA	USB bus current
22	2	i16LE	mV	Battery output bus voltage
24	2	i16LE	mA	Battery output current
26	2	i16LE	mV	Solar input 1 bus voltage
28	2	i16LE	mA	Solar input 1 current
30	2	i16LE	mV	Solar input 2 bus voltage
32	2	i16LE	mA	Solar input 2 current
34	2	i16LE	mV	Solar input 3 bus voltage
36	2	i16LE	mA	Solar input 3 current
38	1	u8	flag	Battery charging complete
39	1	u8	flag	Charger power good
40	4	u32LE	ms	OBC time
44	6	3 × i16LE	raw counts	Magnetometer X, Y, Z
50	2	i16LE	0.1 °C	Barometer temperature
52	4	float32	0.01 hPa	Barometer pressure
56	4	float32	m	Barometric altitude
60	6	3 × i16LE	raw counts	Accelerometer X, Y, Z
66	6	3 × i16LE	raw counts	Gyroscope X, Y, Z
72	4	float32	degrees	GPS latitude
76	4	float32	degrees	GPS longitude
80	4	float32	m	GPS altitude
84	4	float32	km/h	GPS speed
88	4	float32	degrees	GPS course
92	4	float32	dimensionless	GPS horizontal dilution of precision
96	1	u8	count	GPS satellite count
97	1	u8	status	GPS fix

Offset	Size	Type	Units	Meaning
98	12	$3 \times \text{float32}$	degrees	Orientation X, Y, Z
110	2	u16LE	code	OBC error; not exposed by the current UI parser
112	1	u8	version	OBC firmware version
113	4	float32	dBm	Receiver-appended radio RSSI
117	4	float32	dB	Receiver-appended radio SNR

EPS power values shown by the UI are calculated from the corresponding bus-voltage and current fields; they are not transmitted variables. Magnetometer azimuth is likewise calculated by the UI from the X and Y samples. The packed layout has changed between firmware versions. This table describes the current 121-byte V8 packet; consumers should bind their parser to the matching firmware structure in `Common/Inc/telem.h`. No response to this packet is expected.

5.2.2 OBC Information Response (Port 0x12)

Direction: Downlink, OBC STM to host.

This is the solicited response to OBC Information Request. The complete frame begins `12 1A`; its 26-byte payload is:

Offset	Size	Type	Units	Meaning
0	4	u32LE	version	STM/OBC software version
4	8	$8 \times \text{u8}$	ASCII	STM Git hash
12	1	u8	flag	STM dirty build when nonzero
13	4	u32LE	version	Arduino software version
17	8	$8 \times \text{u8}$	ASCII	Arduino Git hash
25	1	u8	flag	Arduino dirty build when nonzero

The Arduino portion is cached by the OBC and does not prove that the Arduino is currently reachable. An all-zero Arduino hash means no current response has been cached.

5.2.3 OBC Legacy Camera Data (Port 0x13)

Direction: Downlink, OBC STM to host.

Older firmware streams this solicited response after OBC Camera Request. Its 244-byte payload is `xBlock:u8, yBlock:u8, pixels[121]:u16LE`; each RGB565 block is 11 by 11 pixels. A frame begins `13 F4`. Newer firmware does not emit this packet.

5.2.4 OBC Flash Settings Response (Port 0x14)

Direction: Downlink, OBC STM to host.

This is the solicited response to OBC Flash Settings Request. Its payload is `settings:u8`; bit 0 controls startup sound. `14 01 01` reports enabled.

5.2.5 OBC JPEG Camera Data (Port 0x15)

Direction: Downlink, OBC STM to host.

Current firmware streams this solicited packet after OBC Camera Request. Each frame begins `15 F2` and contains `blockIndex:u16LE, jpegData[240]:u8`. There is no image ID, total block count, exact final-block length, or image checksum. The host must tolerate interleaved telemetry and detect completion from the JPEG data itself.

5.3 OBC Arduino and Add-ons

The following downlinks originate in the OBC Arduino or an attached add-on and are forwarded through the OBC STM and USB receiver.

5.3.1 User Telemetry (Port 0x11)

Direction: Downlink, OBC Arduino to host.

This asynchronous packet is a packed array of little-endian **float32** values; its payload length must be divisible by four. Values 1.0 and 2.0 produce `11 08 00 00 80 3F 00 00 00 40`.

5.3.2 Arduino Calibration Status (Port 0x80)

Direction: Downlink, OBC Arduino to host.

This asynchronous progress event follows Arduino Start Calibration. Its payload is `status:u8`: 80 01 01 means gyro complete and 80 01 02 means magnetometer complete. It is not a generic command ACK.

5.3.3 Arduino User-Data Name (Port 0x82)

Direction: Downlink, OBC Arduino to host.

This asynchronous metadata packet contains `index:u8`, `name[]:u8`; the name is not null-terminated. Index 0 named “TEMP” is 82 05 00 54 45 4D 50. No response is expected.

5.3.4 Arduino User-Data Item Count (Port 0x83)

Direction: Downlink, OBC Arduino to host.

This asynchronous metadata packet contains `count:u8`; 83 01 04 announces four values. The current V2 UI declares but does not dispatch it. No response is expected.

5.3.5 Arduino Enabled Add-ons Response (Port 0x86)

Direction: Downlink, OBC Arduino to host.

This is the solicited response to Arduino Get Add-ons Request. Its payload is eight `u8` bitmap bytes; ID n uses byte $n/8$, bit $n \bmod 8$. With only IDs 1 and 2 enabled, the frame is 86 08 06 00 00 00 00 00 00 00.

5.3.6 Arduino Add-on Telemetry (Port 0x87)

Direction: Downlink, add-on to host through the OBC Arduino and STM.

This asynchronous packet contains `addonId:u8`, `telemetry[]:u8`. A reaction-wheel packet begins 87 13 01 and has 18 bytes after the ID; an Advanced Sensor packet begins 87 19 02 and has 24. Device layouts are in Section 7. No response is expected.

5.3.7 Arduino Add-on Status (Port 0x88)

Direction: Downlink, add-on to host through the OBC Arduino and STM.

This packet contains `addonId:u8`, `status[]:u8`. It can be solicited by Arduino Get Add-on Status Request or asynchronous. For Advanced Sensor ID 2, its shape is 88 04 02 `sdStatus micStatus radiationStatus`.

5.3.8 Arduino Add-on Information Response (Port 0x89)

Direction: Downlink, OBC Arduino to host.

This is the solicited response to Arduino Add-on Information Request. It contains repeated `addonId:u8`, `version:u16LE` records. For example, 89 06 01 02 00 02 03 00 reports versions 2 and 3 for add-ons 1 and 2.

5.4 Multi-Source Status

Status port 0xF0 can originate at more than one component, so it is grouped separately from component-specific downlinks.

5.4.1 RASCube Status (Port 0xF0)

Direction: Downlink, receiver, OBC STM, or OBC Arduino to host.

This asynchronous packet carries a status source and source-specific data. Receiver status is `source:u8=02`, `status:u16LE`; F0 03 02 01 00 reports receiver status bit 0. Sources 0x00 and 0x01 identify the Arduino and STM OBC, but their additional encoding is not defined by the current UI. Receiver bits are listed in Section 9.1.

6 User Telemetry and Custom Commands

The term “user data” in the current Arduino API means Arduino-to-host telemetry. It does not provide a host-to-Arduino custom command mechanism.

Arduino API	Port	Wire payload
<code>RASCubeV2_UserData_Send(values,n)</code>	0x11	<code>n</code> packed <code>float32</code> values
<code>RASCubeV2_UserData_SetName(index,name)</code>	0x82	<code>index:u8</code> followed by non-null-terminated name bytes
<code>RASCubeV2_UserData_SetNumItems(n)</code>	0x83	One <code>u8</code> item count
<code>RASCubeV2_UserData_TransmitNow()</code>	—	Local timing query; sends no packet itself

Keep names to 32 bytes or fewer because the OBC logger stores at most 32 bytes per name. A radio endpoint can carry at most 61 float32 values after framing, although the intended UI/logging use is smaller and 16 or fewer is the safer operational range.

For inbound commands, an unknown port follows this path:

receiver forwards → OBC forwards → Arduino default handler discards the payload.

The public-looking Arduino receive queue is not functional because the line that would enqueue unknown packets is commented out and there is no callback registration implementation. Consequently, sending a custom port such as 0x90 from Python does not make it available to a user sketch and produces no response. Supporting custom inbound commands requires a firmware change and a separately assigned command ID; it must not be assumed by host software using the current firmware.

7 Add-ons

7.1 Common Structure

Add-on commands use Arduino port 0x86 with this payload:

`addonId:u8, subcommandId:u8, subcommandPayload...`

The add-on IDs are:

ID	Add-on
1	Reaction Wheel
2	Advanced Sensor Module (called Radiation Sensor in UI/Arduino code)
3	Environmental Sensor

Discovery, telemetry, status, and version ports are direction-dependent:

Port	Direction and purpose	Payload
0x85	Host to Arduino: request enabled add-ons	Dummy byte
0x86	Arduino to host: enabled add-ons	Eight-byte bitmap; ID n is byte $n/8$, bit $n \bmod 8$
0x86	Host to Arduino: add-on command	Add-on ID, subcommand, command data
0x87	Arduino to host: add-on telemetry	Add-on ID followed by device telemetry
0x87	Host to Arduino: request status	Dummy byte
0x88	Arduino to host: status	Add-on ID followed by status bytes
0x88	Host to Arduino: request versions	Dummy byte
0x89	Arduino to host: versions	Repeated <code>addonId:u8, version:u16LE</code> records

If no add-ons are enabled, the version request produces no response. Add-on command functions complete after queuing/writing; they do not wait for end-to-end execution. The enabled-add-on request returns the bitmap cached by Arduino initialization or `RASCubeV2_EnableAddon`; it does not rescan the I²C bus.

7.2 Advanced Sensor Module

7.2.1 Host Commands

The Advanced Sensor Module is add-on ID 2. It currently has one remote subcommand:

Subcommand	Data	Meaning
0x00	0x00	Start WAV recording
0x00	0x01	Stop WAV recording

Complete USB frames are therefore:

Start recording: 86 03 02 00 00

Stop recording: 86 03 02 00 01

There is no command ACK. A later status response can show the SD recording bit, but it cannot prove which request caused the state.

7.2.2 Telemetry and Status

Advanced Sensor telemetry arrives on port 0x87 with add-on ID 2 followed by this 24-byte structure. Offsets begin at the first byte after the add-on ID:

Offset	Size	Type	Units	Meaning
0	4	u32LE	ms	Arduino <code>millis()</code> timestamp
4	2	u16LE	0.01 dBA	Instantaneous A-weighted sound level
6	2	u16LE	0.01 dBA	Slow A-weighted sound level
8	2	u16LE	0.01 dBA	Fast A-weighted sound level
10	2	u16LE	0.01 dB SPL	Instantaneous sound-pressure level
12	2	u16LE	0.01 dB SPL	Slow sound-pressure level
14	2	u16LE	0.01 dB SPL	Fast sound-pressure level
16	4	u32LE	count	Cumulative radiation events
20	2	u16LE	0.02 CPM	Slow radiation rate
22	2	u16LE	0.02 CPM	Fast radiation rate

Status arrives on port 0x88 as 02, `sdStatus`, `micStatus`, `radiationStatus`. The status bits are:

Byte	Bit	Meaning
SD	0	Hardware good
SD	1	Filesystem good
SD	2	Initialized
SD	3	Recording
Microphone	0	Initialized
Microphone	1	Hardware fault
Microphone	2	Recording or queue fault
Radiation	0	Hardware present
Radiation	1	Initialized
Radiation	2	Hardware fault

7.2.3 I²C Register Interface

The STM Advanced Sensor Module is an I²C slave at 7-bit address 0x08.

Register	Name	Access	Bytes	Contents
0x00	Version	Read	2	Firmware version, u16LE; inspected source reports 2
0x01	Microphone data	Read	12	Six u16LE sound values, each scaled by 100
0x02	Radiation data	Read	8	Event count plus slow and fast CPM values
0x03	Record command	Write	1	0x00 start, 0x01 stop
0x04	Status	Read	3	SD, microphone, and radiation status bytes

The add-on manager requests a 20 Hz update, but the Advanced Sensor Arduino wrapper ignores that argument and retains a 100 ms period, giving approximately 10 Hz polling. The radiation CPM calculations update internally at approximately 5 Hz. I²C errors are generally not propagated to the UI, and the polling loop can mark data as new even after a failed read.

7.3 Environmental Sensor

The Environmental Sensor is add-on ID 3. The current UI implements telemetry parsing but no host subcommands or status payload. Telemetry uses the Arduino Add-on Telemetry port and begins 87 17 03; the 22 bytes after the add-on ID are shown below. Offsets begin at the first byte after the add-on ID:

Offset	Size	Type	Units	Meaning
0	4	u32LE	ms	Arduino timestamp
4	2	u16LE	raw counts	SHT20 temperature
6	2	u16LE	raw counts	SHT20 relative humidity
8	2	u16LE	raw counts	CO ₂ -sensor temperature
10	2	u16LE	raw counts	CO ₂ -sensor relative humidity
12	2	u16LE	ppm	CO ₂ concentration
14	2	u16LE	raw counts	VOC sample
16	2	u16LE	raw counts	NOx sample
18	2	i16LE	dimensionless	VOC index
20	2	i16LE	dimensionless	NOx index

No response to an Environmental Sensor telemetry packet is expected.

7.4 Reaction Wheel

7.4.1 Host-Level Control

The UI-facing reaction-wheel interface is add-on ID 1. These commands control an Arduino-side orientation PID loop; they are not direct mappings to every wheel I²C register.

Subcommand	Name	Data	Complete add-on payload
0x00	Set enable	enable :u8	01 00 enable
0x01	Set target orientation	target :i32LE	01 01 target [4]
0x02	Set PID gains	Three float32 : P, I, D	01 02 P [4] I [4] D [4]

For example, target orientation 90 is:

```
86 06 01 01 5A 00 00 00
```

The Arduino loop normally runs at 8 Hz, limits its commanded wheel-speed value to ± 6000 , and writes the resulting signed 16-bit value to wheel register 0x01. Disabling the control loop immediately commands speed zero. None of these commands returns an ACK.

Wheel telemetry arrives on port 0x87 with ID 1 and an 18-byte structure. Offsets begin at the first byte after the add-on ID:

Offset	Size	Type	Units	Meaning
0	4	u32LE	ms	Arduino timestamp
4	2	i16LE	degrees	Measured orientation
6	2	i16LE	degrees	Target orientation
8	2	i16LE	controller units	P contribution
10	2	i16LE	controller units	I contribution
12	2	i16LE	controller units	D contribution
14	2	i16LE	firmware-dependent	Target wheel speed
16	2	i16LE	firmware-dependent	Measured wheel speed

All two-byte values after time are signed little-endian integers. The C structure declares time as signed while the UI reads it as unsigned; values agree until the sign bit is reached.

7.4.2 Expanded I²C Contract

The shared Arduino library expects a wheel at 7-bit address 0x04 with this register set:

Register	Name	Access used	Bytes	Encoding
0x00	Version	Read	2	u16LE
0x01	Target speed	Read/write	2	i16LE
0x02	Real speed	Read	2	i16LE
0x03	Speed-to-torque ramp	Read/write	10	Two speeds and three Q15 torque values
0x04	Speed-to-acceleration ramp	Read/write	6	End speed, minimum acceleration, maximum acceleration
0x05	Alignment parameters	Read/write	8	time:u32LE , Q15 q- and d-axis values
0x06	Phase currents	Read	6	Three u16LE samples
0x07	Phase duty cycles	Read	6	Three u16LE values

The Arduino header labels target and real speed as $\text{RPM} \times 10$, but the wheel firmware variants do not consistently implement that unit. Q15 conversion is `value * 32767` without clamping. Direct use of the configuration registers must therefore be tied to a known wheel image rather than inferred from the common header alone.

8 Writing an External Client

8.1 Host Requirements

A robust host should:

- open the CDC port and assert DTR;
- wait briefly for the receiver connection reset, then send the serial-number filter again;
- preserve parser state across partial reads;
- continuously consume asynchronous frames;
- route responses by port and expected payload shape, not by arrival order;
- use one dummy byte for no-parameter requests;
- restrict radio-bound payloads to 247 bytes;
- apply explicit timeouts without assuming a timeout means a command failed;
- avoid concurrent requests that expect the same response port.

8.2 Direct Radio Implementations

A direct SX1262/SDR client must construct the five-byte address header and endpoint records itself, configure the exact frequency/modulation/sync word, enable LoRa CRC, and transmit during the OBC's finite post-transmit receive window. No application CRC bytes should be appended. The official USB receiver handles this timing and is the recommended interface for custom host software.

9 Limitations and Troubleshooting

9.1 Asynchronous Receiver Status

The receiver emits port 0xF0 with payload `source=02, status:u16LE`. Status bits are:

Bit	Mask	Meaning
0	0x0001	Radio hardware initialized successfully
1	0x0002	USB packet dropped
2	0x0004	Radio packet dropped
3	0x0008	Radio waiting for RX
4	0x0010	Radio packet received indication
5	0x0020	Radio packet transmitting indication